

Exercises on Java monitors and conditional synchronization
NOT ASSESSED

Instructions: This exercise sheet is not assessed - it is to help you to understand the material, and to challenge what you think you know. Solutions for the following questions will be provided.

Collaboration: You are permitted to work with your classmates. However, you are likely to get more out of this if you attempt the answers first, before discussing them.

1. Consider the following class.

```
public class SecretHolder {
    private int secret = 0;
    private final MAX_VALUE = 5;

    private synchronized void checkSecret(){
        if (secret < 0 || secret > MAX_VALUE) {
            throw new AssertionError("Secret out of bounds!");
        }
    }

    public synchronized void upOne() throws InterruptedException {
        if (secret == MAX_VALUE) {
            wait();
        }
        secret++;
        checkSecret();
        notify();
    }

    public synchronized void downOne() throws InterruptedException {
        if (secret == 0) {
            wait();
        }
        secret--;
        checkSecret();
        notify();
    }

    public synchronized void upTwo() throws InterruptedException {
        while (secret >= MAX_VALUE - 1) {
            wait();
        }
        secret = secret + 2;
    }
}
```

```

        checkSecret();
        notifyAll();
    }

    public synchronized void downTwo() throws InterruptedException {
        while (secret <= 1) {
            wait();
        }
        secret = secret - 2;
        checkSecret();
        notifyAll();
    }
}

```

Answer the following questions about a concurrent system with several threads that call methods on the same `SecretHolder` instance `I`. We denote any state of the system such that no thread can make any more progress (i.e., it is terminated or indefinitely blocked) as *final state*. Additionally, we write $T=\{A_1, \dots, A_n\}$ to indicate a thread `T` which starts, executes actions $A_1, \dots, A_n$ and then terminates.

- (a) Suppose that the system concurrently runs two threads `A` and `B` such that $A = \{I.downOne()\}$, and $B = \{I.upOne()\}$.
How many final states does the system admit? What are the values of `I.secret` in each final state of the system?
- (b) As in the previous question, suppose that the system concurrently runs two threads `A` and `B` such that $A = \{I.downOne()\}$, and $B = \{I.upOne()\}$.
Can an `AssertionError` be thrown at runtime?
- (c) Consider now the case in which the system concurrently runs four threads `A`, `B`, `C` and `D`, such that $A = \{I.downOne()\}$, $B = \{I.downOne()\}$, $C = \{I.downOne()\}$ and $D = \{I.upTwo()\}$.
Can an `AssertionError` be thrown at runtime?
- (d) Suppose that at a given time, `I.secret` is set to 5 and the last three threads to be executed are $A = \{I.downOne()\}$, $B = \{I.upOne()\}$, and $C = \{I.upTwo()\}$.
Is `I.secret = 4` in any final state of the system?
- (e) Consider any number of threads, such that some of them call `I.upTwo()` once and then terminate, while the others call `I.downTwo()` once and then terminate.
Can an `AssertionError` be raised at runtime? If yes, describe a case where the `AssertionError` is raised. Otherwise, motivate why `I.secret` is always between 0 and `MAX_VALUE` every time `checkSecret()` is called.